方法

GRPO

GRPO[9] is the RL algorithm we use to train DeepSeek-R1-Zero and DeepSeek-R1. The algorithm was originally proposed to simplify the training process of proximal policy optimization (PPO) and reduce its resource consumption[31], ; PPO is widely used in the RL stage of LLMs[32]. The GRPO procedure is shown in Extended Data Fig. 2.

For each question q, GRPO samples a group of outputs $\{o_1, o_2, \ldots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$, and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right)A_i\right) - \beta\mathbb{D}_{KL}(\pi_\theta||\pi_{\text{ref}})\right), \quad (1)$$

$$\mathbb{D}_{\text{KL}}(\pi_\theta||\pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

Here, π_{ref} is the reference policy, ϵ and β are hyperparameters, and A_i is the advantage, calculated using a set of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to each group of outputs:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}. \quad (3)$$

We provide a comparison of GRPO and PPO in Section 1.3 of the Supplementary Information.

Reward Design

Rewards are the source of the training signal and determine the direction of RL optimization. For DeepSeek-R1-Zero, we use rule-based rewards to provide precise feedback for data in mathematics, programming, and logical reasoning. For DeepSeek-R1, we extend this approach by combining rule-based rewards for reasoning-oriented data with model-based rewards for general-purpose data, thereby enhancing the adaptability of the learning process across different domains.

Rule-based rewards. Our rule-based reward system primarily comprises two types of rewards: accuracy rewards and format rewards.

Accuracy rewards assess whether responses are correct. For example, for mathematics problems with determinate results, the model is required to provide the final answer in a specified format (e.g., enclosed in a box), enabling reliable rule-based verification of its correctness. Similarly, for competitive programming prompts, a compiler can be used to evaluate the model's responses against a set of predefined test cases, thereby generating objective feedback on correctness.

The format reward complements the accuracy reward model by enforcing specific formatting requirements. Specifically, the model is incentivized to enclose its reasoning process within designated tags, namely <think> and </think>.. This ensures that the model's thinking process is clearly marked, thereby enhancing interpretability and facilitating subsequent analysis.

$$\text{Reward}_{\text{rule}} = \text{Reward}_{\text{acc}} + \text{Reward}_{\text{format}} \quad (4)$$

The accuracy reward and format reward are combined with equal weights. Notably, we do not apply neural reward models to reasoning tasks, whether outcome-based or process-based. This decision is based on our observation that neural reward models are susceptible to reward hacking during large-scale RL. Furthermore, retraining such models requires substantial computational resources and increases the complexity of the training pipeline, thereby making the overall optimization process more complex.

Model-based rewards. For general data, we use a reward model to capture human preferences in complex and nuanced scenarios. Following the DeepSeek-V3 pipeline, we use preference pairs and training prompts with similar distributions. For helpfulness, we focus solely on the final summary, ensuring that the evaluation emphasizes the response's usefulness and relevance to the user while minimizing interference with the underlying reasoning process. For harmlessness, we evaluate the model's entire response, including the reasoning process and summary, to identify and mitigate any potential risks, biases, or harmful content that may arise during generation.

Helpfulness reward model. When training the helpfulness reward model, we first prompt DeepSeek-V3 using the Arena-Hard prompt format listed in Section 2.2 of the Supplementary Information to generate preference pairs, each containing a user query and two candidate responses. For each preference pair, we query DeepSeek-V3 four times, randomly assigning the responses as Response A or Response B to mitigate position bias. The final preference scores are determined by averaging four independent judgments, and only preference pairs with a score difference (Δ) greater than 1 are retained to ensure meaningful differences. In addition, to minimize length-related bias, we ensure that the chosen and rejected responses are of similar lengths across the entire dataset. In total, we compiled 66,000 data pairs to train the reward model. The prompts used in this dataset are all non-reasoning questions, sourced from publicly available open-source datasets or from users who explicitly consented to sharing their data to improve the model. Our reward model architecture is consistent with DeepSeek-R1, with an additional reward head for predicting scalar preference scores.

$$\text{Reward} = \text{RM}(\text{Response}_A, \text{Response}_B)_{\text{helpful}} \quad (5)$$

The helpfulness reward model is trained for one epoch on the training dataset with a batch size of 256 and a learning rate of 6×10^{-6}. The maximum sequence length during training is set to 8,192 tokens, while no explicit length limit is imposed during reward model inference.

Safety reward model. To evaluate and improve the model's safety, we curated a dataset containing 106,000 prompts and labeled the model-generated responses as 'safe' or 'unsafe' according to predefined safety guidelines. Unlike the pairwise loss used by the helpfulness reward model, the safety reward model is trained using a pointwise approach to distinguish safe responses from unsafe ones. The training hyperparameters are the same as those of the helpfulness reward model.

$$\text{Reward}_{\text{safety}} = \text{RM}_{\text{safety}}(\text{Response}) \quad (6)$$

For general queries, each instance is classified as belonging to either the safety dataset or the helpfulness dataset. The general reward $\text{Reward}_{\text{general}}$ assigned to each query corresponds to the respective reward defined in the dataset to which it belongs.

Training Details

Training details for DeepSeek-R1-Zero. To train DeepSeek-R1-Zero, we set the learning rate to 3×10^{-6}, the Kullback–Leibler (KL) coefficient to 0.001, and the rollout sampling temperature to 1. For each question, we sample 16 outputs, with a maximum length of 32,768 tokens before step 8.2k and 65,536 tokens thereafter. Consequently, both the performance and response length of DeepSeek-R1-Zero show a significant jump at step 8.2k. Training continues for a total of 10,400 steps, corresponding to 1.6 training epochs. Each training step includes 32 distinct questions, so the training batch size per step is 512. Every 400 steps, we replace the reference model with